GENETIC ALGORITHMS & EVOLUTION STRATEGIES

Filipe Coutinho
s3801454
f.z.lopes.de.coutinho@umail.leidenuniv.nl

Myriana Miltiadous
s3699463
m.miltiadous@umail.leidenuniv.nl

December 26, 2023

1 Introduction

In the area of mathematical computing, optimizing combinatorial issues has been a primary focus, with applications spanning from statistical physics to communication systems. In this report, we address two challenging problems: the Low Auto-correlation Binary Sequences (LABS) and the Ising Model. Both problems pose unique challenges in the context of evolutionary algorithms, and the objective here is to employ a Genetic Algorithm (GA) and an Evolution Strategy (ES) to explore their solution spaces. Both Grid Search and Sequential Model-Based Optimization are utilized to find the best-performing parameters for the problems, for each of the two algorithms, which are reported in the following sections, with their respective averaged found fitness.

1.1 Problems Definition

1.1.1 Low Auto-correlation Binary Sequences (LABS)-F18

The Low autocorrelation Binary Sequences (LABS) problem poses a non-linear objective function over a binary sequence space intending to maximize the reciprocal of the sequence's autocorrelation: $x \in \{0,1\}^n$
LABS: x $\rightarrow \frac{n^2}{2\sum_{k=1}^{n-1}(\sum_{i=1}^{n-k} s_i s_{i+k})^2=1}$, where $s_i = 2x_i - 1$

Goal : Find $x \in \{0,1\}^n$, such that $E(x) = \sum_{k=1}^{n-1}(\sum_{k=1}^{n-k} y_i y_{i+k})^2 \rightarrow$ min, where $y_i = 2x_i - 1$

Alternative Formulation: Fitness: $F(x) = \frac{n^2}{2E(x)} \rightarrow$ max

Theory indicates that $\lim_{n\rightarrow\infty} \arg\max F(x) \approx 12.32$

1.1.2 The ISING Model-F19

The classical Ising model considers a set of spins placed on a regular lattice $G = ([n], E)$, where each edge $(i, j) \in$ E is associated with an interaction strength J_{ij}. Given a configuration of n spins, $S = (s_1, ..., s_n) \in \{-1, 1\}^n$ this problem possesses a quadratic function, representing the system's energy and depending on its structure J_{ij}. Assuming zero external magnetic fields and using x_i =$(s_i + 1)/2$ and a unit interaction strength, the following pseudo-Boolean maximization problem is obtained:
ISING : x $\rightarrow \sum_{i,j\in E}(x_i x_j + (1 - x_i)(1 - x_j))$

Again for this problem, the goal is to maximize the fitness function For this report, the instance of lattice is considered to be the one-dimensional ring (F19).

2 Algorithms

In this section both the algorithms (genetic algorithm & evolution strategy) are described with their respective pseudocode provided, along with the report of the parameters that manage to achieve the biggest average fitness value. All the exact code used is additionally provided in the Appendix.

2.1 Genetic Algorithm

Algorithm 1: Genetic Algorithm Pseudocode

Input : Problem instance: $problem$
Population size: pop_size
Budget: $budget$ (default: 5000)
Crossover probability: $crossover_probability$
n for n-points crossover type: n_points
k for tournament-k selection type: $tournament_k$
Mutation rate: $mutation_rate$
Mutation strength for binomial mutation: $mutation_strength$
Optimum for a specific problem: $optimum$
Dimension: $dimension$

Termination : The algorithm terminates when problem evaluations exceed the budget

```
Initialize f_opt with sys.float_info.min
Initialize optimum based on problem
for i = 1 to pop_size do
    Create random individual: parent[i] ← Random binary string of length dimension
    Evaluate fitness parent_f[i] of parent[i]
end
while Termination criteria not met do
    Selection to population;
    Crossover to population;
    Mutation to population; ;                                   // Updated population (parent)
    for i = 0 to pop_size do
        Evaluate fitness parent_f[i] of parent[i]
        if parent_f[i] > f_opt then
            f_opt ← parent_f[i]
            x_opt ← parent[i].copy()
        end
        if f_opt ≥ optimum or problemevaluations >= budget then
            break
        end
    end
end
Return f_opt
```

For implementing the Genetic algorithm for both F18 and F19 problems we used the pseudocode for the genetic algorithm illustrated in Algorithm 1, which provides the general idea of its functioning. The algorithm begins with the initialization of a population of individuals, where each individual is represented as a binary vector. The fitness of each individual, which is the value we want to maximize, is evaluated based on the given optimization problem(for us either F18 or F19). The algorithm then enters a loop where it continues to evolve the population through genetic operations such as mutation, crossover, and selection. The main loop runs until a specified budget of function evaluations (budget) is exhausted. The algorithm keeps track of the best solution (x_opt) and its corresponding fitness value (f_opt) throughout the optimization process. If the algorithm finds a solution with a fitness value equal to or greater than the specified optimum (which for F18 we specified it to be 8 and for F19 50) or exceeds the budget, it breaks out of the loop and prints the best fitness value and the corresponding solution. The final best fitness value is returned to be added to a sum and after finishing the specified procedure for 20 (in our case) times the average is calculated. The pseudocode is not illustrated but two types per operation were tested to find the ones that yield the maximum fitness, and for each choice, a function was created. For selection, the choices were either proportional or tournament k, for crossover either n-points or uniform and finally for mutation either bitflip or binomial. Following the exact functioning of each of the

methods is given
Proportional selection The fitness values of the population's individuals are being utilized by this function. It computes a cumulative probability distribution and modifies fitness values (input) to avoid division by zero using a proportional fitness scaling. The probability that each person will be chosen as a parent depending on their fitness is represented by this distribution. Then, the function stochastically selects people, giving preference to those who are more fit, using random numbers. By guaranteeing that those with higher fitness have a higher probability of procreating, this mechanism encourages the population to evolve towards more ideal solutions.
Tournament-k selection Three arguments are passed into the function: the size of the tournament (tournament_k), the parent population (parent), and their matching fitness values (parent_f). The tournament size is the number of individuals randomly selected from the population for comparison. The function then iterates through the population, conducting tournaments by randomly selecting a subset of individuals, comparing their fitness values, and choosing the individual with the highest fitness as the winner. After then, a copy of the winner is added to the chosen parent group. Every member of the population goes through the same procedure, which increases the variety of parents chosen and improves the genetic algorithm's search for answers.
N-points crossover The function chooses n crossover points, where n is defined by a $npoints$ argument, if the randomly produced number is less than the given crossover probability. Then, by calling another function at each of these points, creating diverse offspring. The single point crossover function takes two parent individuals (p1 and p2) and a crossover point x, and it performs a single-point crossover by swapping genetic material beyond the specified point. It then returns the resulting offspring.
Uniform crossover The function works by using a given crossover_probability to determine whether a crossover event should happen. A randomly generated number from a uniform distribution between 0 and 1 is compared to the given crossover probability in order to make this determination. The function iterates through each bit of the parent vectors in order to determine whether a crossover event will occur. A new random number is created for every bit, and if it is less than 0.5, the associated bits of p1 and p2 are switched.
Bitflip mutation: This function takes a binary vector p as input and iterates through each bit, deciding whether to mutate it according to a specified mutation_rate. A randomly generated number from a uniform distribution between 0 and 1 is compared with the given mutation rate to determine the outcome. The bit mutates when the condition is satisfied, moving from 0 to 1 or from 1 to 0.
Binomial mutation: The function takes an individual as a binary vector and, given a probability (mutation_probability), conducts bit-wise mutation. Every bit has a random number created for it; if this number is smaller than the mutation probability, the bit gets mutated. The mutation entails taking the modulo of two after appending a random value to the original bit from a binomial distribution with parameters (1, mutation_strength). The bit is flipped by this operation. The intensity of the mutation can be adjusted using the mutation strength option. The altered individual that results is then given back,

Employing the aforementioned algorithm with a budget of **5000** and a dimension of **50**, we conducted a comprehensive grid search to identify optimal parameter configurations that maximize the average fitness value across **20** repetitions of the algorithm, for both the F18 and F19 problems. Because of the randomness and for the results to be reproducible, a specific seed was defined for each configuration of parameters.
The search space is displayed for the n-points crossover and tournament-k selection. The values in table 1 were examined with bit flip mutation whereas the values in table 2 were examined with binomial mutation.

Table 1: Grid Search Parameters for Bitflip Mutation

Pop Size	**Crossover Probability**	**Mutation Rate**	**Seed**	**Tournament Size (k)**	**Npoints**
10	0.2	0.02	42	5	5
30	0.5	0.05		20	10
45	0.8			35	

Table 2: Grid Search Parameters for binomial Mutation

Pop Size	**Crossover Probability**	**Mutation Rate**	**Seed**	**Tournament Size (k)**	**Npoints**	**Mutation Strength**
10	0.2	0.02	42	5	5	0.5
30	0.5	0.05		10	10	1.0
	0.8			20	20	

The **best** reported average fitness for **F18** problem was found to be **4.378066451902746**. The **AUC** value for the specific configuration was calculated, to be **0.691102727272727**, by the IOHanalyzer. The corresponding curve is illustrated on the right of Figure 1.

This value was achieved using bitflip mutation, n-points crossover and tournament-k selection with the following parameters: **npoints = 10**, **tournament_k = 35**, **pop_size = 30**, **crossover_prob = 0.5**, **mutation_rate = 0.05**,**seed = 42**.

The **best** reported average fitness for **F19** problem was found to be **48.2**.
The **AUC** value for the specific configuration was calculated, to be **0.800215454545455**, again by IOHanalyzer. The corresponding curve is illustrated on the right of Figure 3
This value was achieved using binomial mutation, n-points crossover and tournament-k selection with the following parameters: **npoints = 20**, **tournament_k = 5**, **pop_size = 10**, **crossover_prob = 0.2**, **mutation_rate = 0.05**,**seed = 42**, **mutation_strength = 0.5**.

2.2 Evolution Strategy

Algorithm 2: Evolutionary Strategy Algorithm Pseudocode

Input : Problem callable, Population size μ, Offspring size λ, Recombination Function $recombinate$
Termination : The algorithm terminates when problem state evaluations + $\lambda \leq$ budget

```
# Create and initialize the population P(0) of mu individuals
population = Population.new(mu, dimension, problem);
while problem state evaluations + lambda_ ≤ budget do
    offspring_population = Population();
    for _ in range(lambda_) do
        # Recombination
        offspring = recombinate(population);
        # Mutation
        offspring = offspring.mutate();
        fitness = offspring.evaluate(problem);
        offspring.set_fitness(fitness);
        offspring_population.add(offspring);
    end
    # Select new population
    population = offspring_population.elitist_selection(mu);
end
```

The algorithm 2, shows the pseudocode for the implementation of the evolution strategy. Although the pseudocode doesn't show the concrete implementation of the recombination and mutation functions it provides a conceptual overview of how the Evolution Strategy operates. The process initiates with the line $Population.new(mu, dimension, problem)$ where the initial population of mu individuals is generated, with each individual being represented as a real-valued vector. Subsequently, in the same function, the fitness of this initial population is also evaluated. The algorithm proceeds through an iterative loop, continually refining the population through evolutionary mechanisms such as recombination and mutation, and selection until the budget is consumed.

Within this loop, we produce an offspring population of λ individuals by executing three operations: recombination, mutation, and selection. The recombination is done by executing the input function $recombinate$. This function is expected to receive a population and return an offspring, and the available options can be consulted in B.2.

We implemented different types of recombination functions such as:

- Global Discrete Recombination
- Global Intermediate Recombination
- Individual Discrete Recombination
- Individual Intermediate Recombination

Since the Individual Recombination functions require two parent-offspring, we devised distinct implementations to offer various types of parent selection, including random parent selection, tournament parent selection, and roulette parent selection; refer to B.1.

Generated the offspring we now need to perform mutation. We fixed the implementation to mutation - individual σ_i (13). Despite experimenting with mutation using a constant σ, the outcomes for the latter were slightly worse.

We evaluated the offspring's fitness and added it to the offspring population. It's important to highlight that the transformation of the solution vector, containing real values, into a bit vector is necessary. This transformation is achieved by encoding a positive value as 1 and 0 otherwise.

As for the selection strategy, we have two options: (λ, μ)-`ES` and $(\lambda + \mu)$-`ES`. Our experiments revealed that the elitist selection strategy tends to perform better, and thus we settled for it. This means that the best μ individuals in the offspring population are selected to be the next iteration's population.

2.2.1 Parameters Search

In the pursuit of optimal parameters for each of the problems $F18$ and $F19$, we engaged in Sequential Model-Based Optimization. This involved the exploration of optimal values for μ, λ, and the most effective recombination function ($recombinate$).

Table 3: Best found parameters for evolution strategy employing SMBO

Problem	μ	λ	$recombinate$	Average Fitness	AUC
F18	10	112	intermediate_recombination_tournament_parent_selection	3.86	0.51
F19	18	111	discrete_recombination_tournament_parent_selection	45.8	0.77

Table 2.2.1 displays the optimal parameters identified through SMBO, accompanied by the corresponding average fitness achievements. It's worth noting that the $recombinate$ function can be referenced in B.2.

3 Experimental Results

3.1 Genetic Algorithm

Except for the configuration of parameters with the best performance, through the gird search, the 5 best configurations for each of the F18 and F19 problems were identified and their respective figures with explanations are going to be presented in this section, along with their settings. All the figures are downloaded from the IOHanalyzer.

3.1.1 F18

For F18 the optimum value in the genetic algorithm was defined to be 8 since it is a value that has been reported in the literature. The goal was to go as close as possible to this value, knowing however that reaching it is relatively difficult. So, the five best average fitness values, that were found (4.38,4.37,4.37,4.35,4.34), were considered sufficient. All the results, using the configurations of parameters reported, for the 5 experiments, are illustrated in the following figures. As it can be observed, all achieved similar results regarding the aggregated empirical cumulative distribution (1) and the average fitness(figure 2). They all used the same budget of 5000 as this was what the algorithm was forced to do.

Table 4: Top 4 (following the best one) configurations based on F18 Fitness. Configurations with rank i corresponds to exp[i] on the F18 plots

$Rank$	Npoints	Tournament Size (k)	Pop Size	Crossover Prob	Mutation Rate	Seed	Mutation Strength
2	5	20	10	0.5	0.05	42	0.5
3	10	5	10	0.8	0.05	42	0.5
4	20	5	10	0.2	0.02	42	1.0
5	20	10	10	0.5	0.05	42	0.5

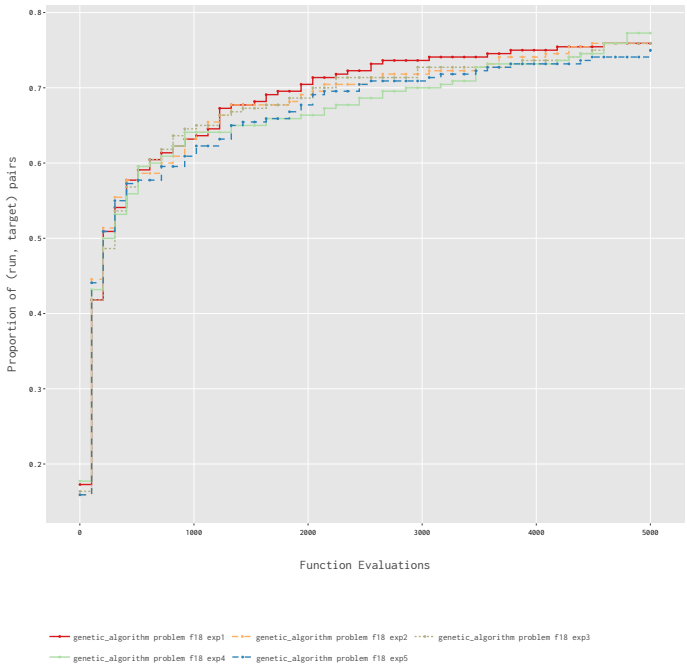

Figure 1: Genetic Algorithm: Aggregated Empirical Cumulative Distribution Curve for the five best configurations for F18. The reported as best one is illustrated with the red line

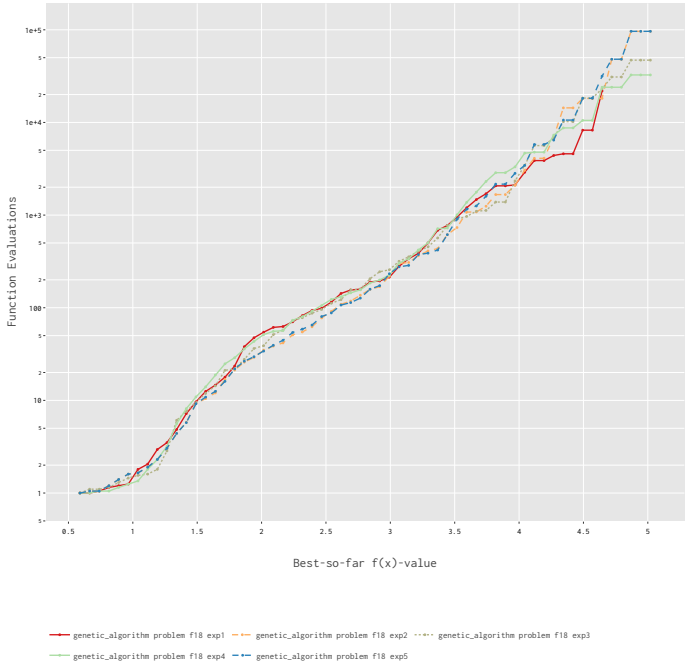

Figure 2: Genetic Algorithm: Expected Runtime Curve (ERT) of 5 best experiments for F18. The reported as best one is illustrated with the red line

3.1.2 F19

For F19 the optimum value in the genetic algorithm was defined to be 50. The goal again was to go as close as possible to this value, knowing of course that reaching it is a difficult task. So, the five best average fitness values, that were found (48.2, 48,48,47.9,47.8), were considered satisfactory. All the results, using the configurations of parameters reported, for the 5 experiments, are illustrated in the following figures. As it can be observed, all achieved similar results regarding the aggregated empirical cumulative distribution (3) and the average fitness(figure 4). Once again, all used the same budget of 5000 as this was what the algorithm was forced to do.

Table 5: Genetic Algorithm: Top 4 (following the best one) combinations based on F19 Fitness. Configurations with rank i corresponds to exp[i+5] on the F19 plots. The reported as best one is illustrated with the yellow line.

Rank	Npoints	Tournament Size (k)	Pop Size	Crossover Prob	Mutation Rate	Seed	Mutation Strength
2	20	10	10	0.8	0.05	42	0.5
3	10	10	30	0.8	0.05	42	1.0
4	20	5	10	0.8	0.02	42	1.0
5	20	10	10	0.5	0.05	42	0.5

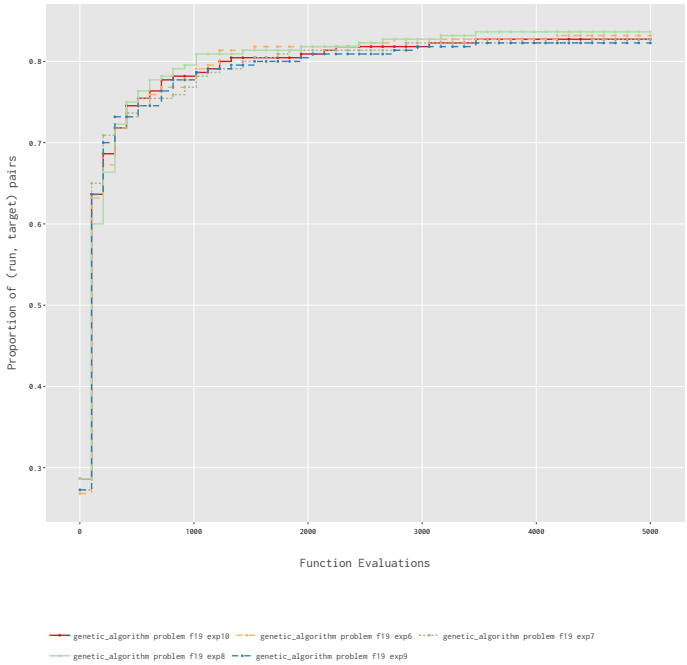

Figure 3: Genetic Algorithm: Aggregated Empirical Cumulative Distribution Curve for the five best configurations for F19. The reported as best one is illustrated with the yellow line fixed target results.

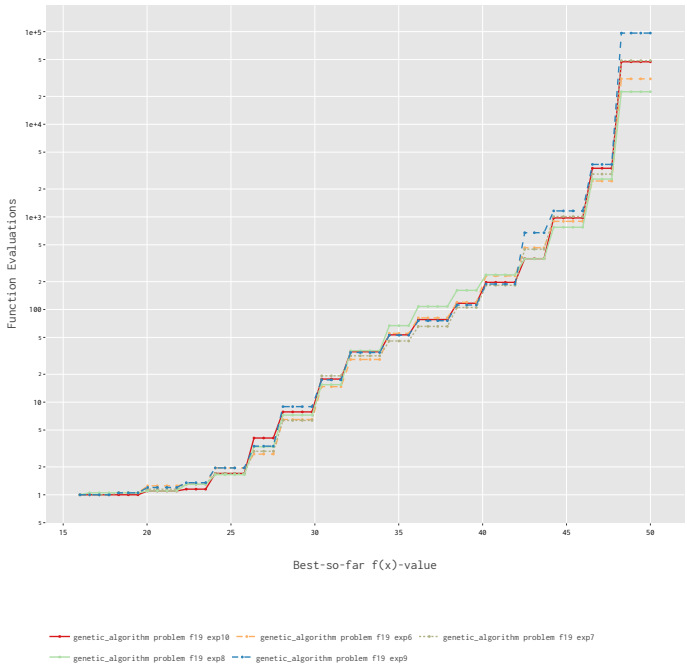

Figure 4: Genetic Algorithm: Expected Runtime Curve (ERT) of 5 best experiments for F19. The reported as best one is illustrated with the yellow line.

3.2 Evolution Strategy

In this section, we will thoroughly examine the optimal configurations for the F18 and F19 problems, which were attained through a comprehensive Sequential Model-Based Optimization (SMBO) approach. The five most promising configurations, each complemented by their corresponding figures and comprehensive explanations, will be showcased. All graphical representations have been retrieved from the IOHanalyzer.

3.2.1 F18

As specified in the section 3.1.1, the established benchmark for the fitness value in the F18 problem is approximately 8. We believe that the results obtained and specified in 2.2.1, which were the best obtained in our experiments, are sufficient. The figures below present the results corresponding to the reported configurations for all five experiments.

Table 6: Top 4 (following the best one) combinations based on F18 Fitness. Configurations with rank i correspond to exp[i+5] on the F19 plots. The reported best one is illustrated with the red line.

Rank	μ	λ	*recombinate*	Average Fitness
2	18	111	discrete_recombination_tournament_parent_selection	3.77
3	15	105	discrete_recombination_tournament_parent_selection	3.74
4	20	140	discrete_recombination_tournament_parent_selection	3.68
5	10	70	discrete_recombination_tournament_parent_selection	3.67

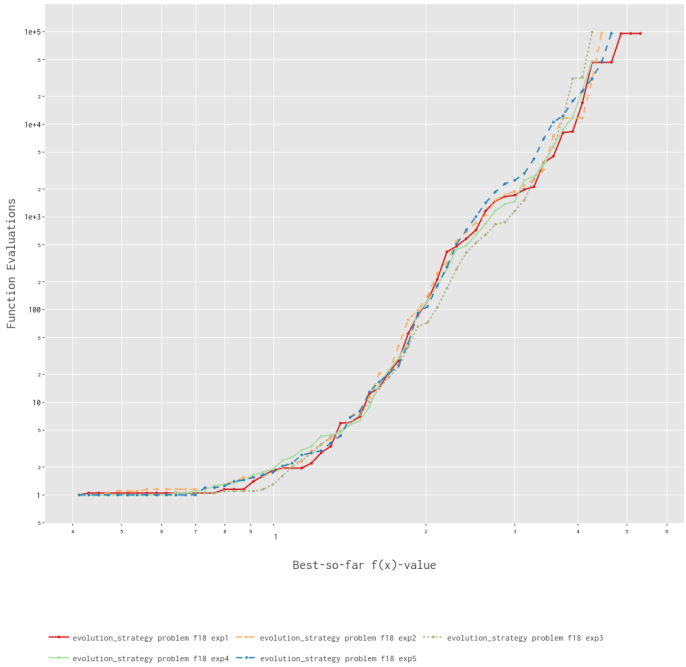

Figure 5: The fixed-target results of 5 best experiments for F18. The reported as best one is illustrated with the red line.

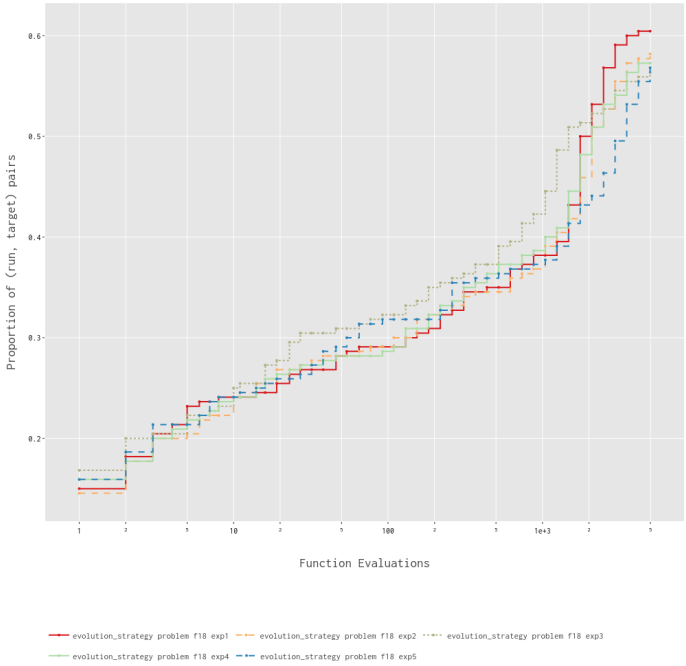

Figure 6: Aggregated Empirical Cumulative Distribution: Single function for the five best configurations for F18. The reported as best one is illustrated with the red line.

3.2.2 F19

Similar to the context discussed in Section 3.1.2, the optimal value for F19 is established at 50. Acknowledging the challenge posed by this state-of-the-art benchmark, our efforts were directed toward obtaining values as close to this

optimum as possible. The outcomes obtained with the identified parameter configurations are depicted in the figures below.

The results using the configurations of parameters found are illustrated in the figures below. The configurations are defined in **??**he best configuration in 2.2.1. The configurations themselves are outlined in Section **??**h the best-performing configuration specified in Section 2.2.1.

Table 7: Top 4 (following the best one) combinations based on F19 Fitness. Configurations with rank i correspond to exp[i+5] on the F19 plots. The reported best one is illustrated with the yellow line.

Rank	μ	λ	$recombinate$	Average Fitness
2	10	402	discrete_recombination_random_parent_selection	45.8
3	15	105	discrete_recombination_tournament_parent_selection	45.6
4	20	140	discrete_recombination_tournament_parent_selection	45.4
5	10	70	discrete_recombination_tournament_parent_selection	44.5

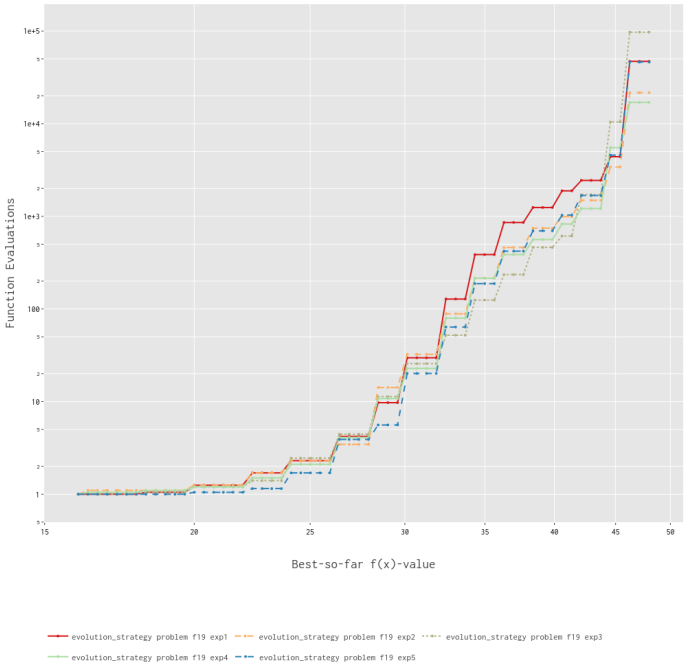

Figure 7: The fixed-target results of 5 best experiments for F19. The reported as best one is illustrated with the red line.

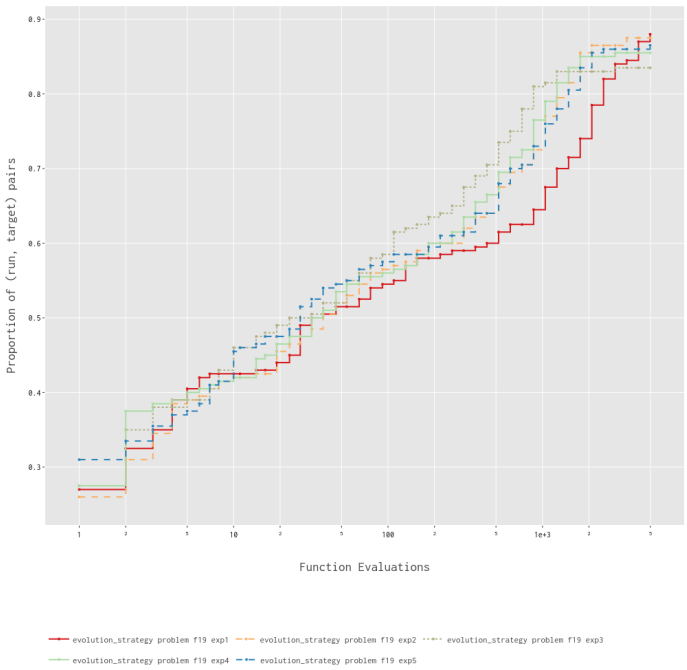

Figure 8: Aggregated Empirical Cumulative Distribution: Single function for the five best configurations for F19. The reported as best one is illustrated with the red line.

4 Discussion and Conclusion

In this section a summary of the results and conclutons of this report is provided:

1) We suggest using bitflip mutation, n-points crossover and tournament-k selection with the following parameters: **number of points = 10**, **k = 35**, **population size = 30**, **crossover probability = 0.5**, **mutation rate = 0.05**, **seed = 42** for the genetic algorithm to solve the problem F18.
2) We suggest using binomial mutation, n-points crossover and tournament-k selection with the following parameters: **number of points = 20**, **k value = 5**, **population size = 10**, **crossover probability = 0.2**, **mutation rate = 0.05**, **seed = 42**, **mutation strength = 0.5** for the genetic algorithm to solve the problem F19.
3) The genetic algorithm benefits from binomial mutation, n-points crossover and tournament-k selection for both F18 and F19 problems, since with those settings it achieved in general better performance compared to the other settings, even though for the F18 problem the best fitness was achieved using bit-flip mutation.
4) For the best-reported values for both F18 and F19 the majority had a population size equal to 10, so that number seems to benefit optimum values for fitness.
5) The genetic algorithm does not seem to benefit either from small or large crossover probabilities since in the best-reported configurations the crossover probabilities vary between 0.2 and 0.8.
6) To solve F18 with an evolution strategy, we recommend using **number of parents = 10**, **number of offspring = 112** and **intermediate recombination with tournament as parent selection**.
7) For problem F19 with evolution strategy, we suggest using **number of parents = 18**, **number of offspring = 111**, and **discrete recombination with tournament as parent selection**.
8) We observe that the evolution strategy benefits from comma selection for solving both F18 and F19 problems
9) In this scope, the genetic algorithm exhibits superior performance compared to evolution strategies.

A Genetic Algorithm - Code

Listing 1: Genetic algorithm implementation with all the mentioned functions-Gives the results for the best configuration for each of F18 and F19.

```
import numpy as np
from ioh import get_problem, logger, ProblemClass
import sys
import itertools
import random



dimension = 50

# Uniform Crossover
def single_point_crossover(p1,p2,x):
    p1_new=np.append(p1[:x],p2[x:])
    p2_new=np.append(p2[:x],p1[x:])
    return p1_new,p2_new


def uniform_crossover(p1, p2, crossover_probability):
    if cross_type == 'uniform':
        if np.random.uniform(0, 1) < crossover_probability:
            for i in range(len(p1)):
                if np.random.uniform(0, 1) < 0.5:
                    t = p1[i]
                    p1[i] = p2[i]
                    p2[i] = t

def npoints_crossover(p1, p2, crossover_probability, npoints):
    if np.random.uniform(0, 1) < crossover_probability:
        cross_points = list(np.random.choice(range(1, 50), npoints, replace=
            False))
        for i in cross_points:
            p1, p2 = single_point_crossover(p1.copy(), p2.copy(), i)
    return p1, p2


# Standard bit mutation using mutation rate p
def mutation(p, mutation_rate):
    for i in range(len(p)) :
        if np.random.uniform(0,1) < mutation_rate:
            p[i] = 1 - p[i]

def binomial_mutation(individual, mutation_probability, mutation_strength):
    mutated_individual = individual.copy()

    for i in range(len(mutated_individual)):
        if np.random.rand() < mutation_probability:
            # Add a random value from a binomial distribution
            mutation_value = np.random.binomial(1, mutation_strength)
            mutated_individual[i] = (mutated_individual[i] + mutation_value) %
                2

    return mutated_individual
```

```
def proportional_selection(parent, parent_f):
    # Plusing 0.001 to avoid dividing 0
    f_min = min(parent_f)
    f_sum = sum(parent_f) - (f_min - 0.001) * len(parent_f)

    rw = [(parent_f[0] - f_min + 0.001)/f_sum]
    for i in range(1,len(parent_f)):
        rw.append(rw[i-1] + (parent_f[i] - f_min + 0.001) / f_sum)

    select_parent = []
    for i in range(len(parent)) :
        r = np.random.uniform(0,1)
        index = 0
        while(r > rw[index]) :
            index = index + 1
        select_parent.append(parent[index].copy())
    return select_parent


def tournamentk_selection(parent, parent_f, tournament_k):
    select_parent = []
    tournament_k = min(tournament_k, len(parent_f))
    pre_select = np.random.choice(len(parent_f), tournament_k, replace=False)
    index = 0
    for i in range(len(parent)) :
        pre_select = np.random.choice(len(parent_f),tournament_k,replace =
            False)
        max_f = sys.float_info.min
        for p in pre_select:
            if parent_f[p] > max_f:
                index = p
                max_f = parent_f[p]
        select_parent.append(parent[index].copy())
    return select_parent



def studentnumber1_studentnumber2_GA(problem,  pop_size, budget = None):
    if problem.meta_data.problem_id == 18:
        optimum=8
    if problem.meta_data.problem_id == 19:
        optimum=50
    if budget is None:
        budget = 5000

    f_opt = sys.float_info.min
    x_opt = None
    parent = []
    parent_f = []
    for i in range(pop_size):

        # Initialization
        parent.append(np.random.randint(2, size = problem.meta_data.
            n_variables))
        parent_f.append(problem(parent[i]))


    while problem.state.evaluations < budget :
        if seletype == 'proportional':
```

```
            offspring = proportional_selection(parent,parent_f)
        elif seletype == 'tournamentk':
            offspring = tournamentk_selection(parent,parent_f,tournament_k)

        if cross_type=='npoints':
            for i in range(0,pop_size - (pop_size%2),2) :
                offspring[i], offspring[i+1] = npoints_crossover(offspring[i
                    ], offspring[i+1], crossover_probability, npoints)
        elif cross_type=='uniform':
            for i in range(0,pop_size - (pop_size%2),2) :
                uniform_crossover(offspring[i], offspring[i+1],
                    crossover_probability)

        if mutatype == 'bitflip':
            for i in range(pop_size):
                mutation(offspring[i], mutation_rate)
        elif mutatype == 'binomial':
            for i in range(pop_size):
                offspring[i] = binomial_mutation(offspring[i], mutation_rate,
                    mutation_strength)



        parent = offspring.copy()
        for i in range(pop_size) :
            parent_f[i] = problem(parent[i])
            if parent_f[i] > f_opt:
                    f_opt = parent_f[i]
                    x_opt = parent[i].copy()
            if f_opt >= optimum or problem.state.evaluations >= budget:
                break

    print(f_opt,x_opt)
    return f_opt

def create_problem(fid: int):

    problem = get_problem(fid, dimension=dimension, instance=1, problem_class=
        ProblemClass.PBO)
    l = logger.Analyzer(
        root=f"dataf{fid}", # the working directory in which a folder named '
            folder_name' (the next argument) will be created to store data
        folder_name=f"experiment{i+1}", # the folder name to which the raw
            performance data will be stored
        algorithm_name=f"genetic_algorithm␣problem␣f{fid}␣exp{i+1}", # name
            of your algorithm
        algorithm_info="Practical␣assignment␣of␣the␣EA␣course",
    )
    # attach the logger to the problem
    problem.attach_logger(l)
    return problem, l



if __name__ == "__main__":



    # best parameter configuration for each problem
```

```
npointss = [10,20]
tournament_ks=[35,5]
pop_sizes=[30,10]
crossover_probabilities=[0.5,0.2]
mutation_rates=[0.05,0.05]
seed=42
mutation_strength= 0.5


for i in range(2): #running best configuration for each of the f18 and f19
    problems
    seletype = 'tournamentk' #'proportional'
    cross_type = 'npoints' #'uniform'


    crossover_probability = crossover_probabilities[i]
    mutation_rate =mutation_rates[i]
    pop_size = pop_sizes[i]

    tournament_k = tournament_ks[i]
    npoints = npointss[i]


    if i>0:
        mutatype='binomial'
    else:
        mutatype = 'bitflip'




    # this how you run your algorithm with 20 repetitions/independent run
    if i<1:
        np.random.seed(seed)

        F18, _logger = create_problem(18)
        #optimum=15
        s_f_opt=0
        for run in range(20):
            print(f'F18, Run:{run}')
            f_opt = studentnumber1_studentnumber2_GA(F18, pop_size)
            s_f_opt=s_f_opt+f_opt
            F18.reset() # it is necessary to reset the problem after each
                independent run
        _logger.close() # after all runs, it is necessary to close the
            logger to make sure all data are written to the folder
        print('F18 fitness average:',s_f_opt/20)

    else:
        np.random.seed(seed)

        F19, _logger = create_problem(19)
        #optimum=50
        s_f_opt2=0
        for run in range(20):
            print(f'F19, Run:{run}')
            f_opt2=studentnumber1_studentnumber2_GA(F19, pop_size)
            s_f_opt2=s_f_opt2+f_opt2
            F19.reset()
```

```
        _logger.close()
        print('F19 fitness average:',s_f_opt2/20)
```

B Evolution Strategies - Code

B.1 Parent Selections

Listing 2: Random Selection Function

```
def random_selection(self, random_state: int = None) -> Individual:
    """
    Selects a random individual from the population.

    This function selects a random individual from the population stored in
        the object.

    Args:
    - random_state (int): Seed for the random number generator. Defaults to
        None.

    Returns:
    - Individual: The randomly selected individual.
    """
    rs = np.random.RandomState(
        random_state) if random_state is not None else np.random
    population_size = len(self.population)
    return self.population[rs.randint(0, population_size)]
```

Listing 3: K-Tournament Selection Function

```
def k_tournament_selection(self, k: int, random_state: int = None) -> List[
    Individual]:
    """
    Select parents using k-tournament selection.

    Args:
    - k (int): The tournament size.
    - random_state (int): Seed for the random number generator. Defaults to
        None.

    Returns:
    - List[Individual]: The selected parents.
    """
    rs = np.random.RandomState(
        random_state) if random_state is not None else np.random
    selected_parents = []
    tournament_size = k
    population_size = len(self.population)

    for _ in range(tournament_size):
        tournament_indices = rs.choice(
            population_size, size=k, replace=False)
        tournament = [self.population[i] for i in tournament_indices]
        tournament_fitness = [ind.fitness for ind in tournament]
        best_individual = tournament[np.argmax(tournament_fitness)]
        selected_parents.append(best_individual)

    return selected_parents
```

Listing 4: Roulette Wheel Selection Function

```
def roulette_selection(self, random_state: int = None) -> Individual:
    """
    Select an individual from the population using roulette wheel selection.

    Args:
    - random_state (int): Seed for the random number generator. Defaults to
        None.

    Returns:
    - Individual: The selected individual.
    """
    rs = np.random.RandomState(
        random_state) if random_state is not None else np.random
    fitness = self.fitness()
    total_fitness = sum(fitness)
    selection_point = rs.uniform(0, total_fitness)

    cumulative_fitness = 0
    selected_index = 0

    for i, ind_fitness in enumerate(fitness):
        cumulative_fitness += ind_fitness
        if cumulative_fitness >= selection_point:
            selected_index = i
            break

    return self.population[selected_index]
```

B.2 Recombination Implementations

Listing 5: Global Discrete Recombination Function

```
def global_discrete_recombination(population: Population) -> Individual:
    return population.global_discrete_recombination()
```

Listing 6: Global Intermediate Recombination Function

```
def global_intermediate_recombination(population: Population) -> Individual:
    return population.global_intermediate_recombination()
```

Listing 7: Discrete Recombination with Random Parent Selection Function

```
def discrete_recombination_random_parent_selection(population: Population) ->
    Individual:
    parent1, parent2 = population.random_selection(), population.
        random_selection()
    offspring = parent1.discrete_recombination(parent2)
    return offspring
```

Listing 8: Discrete Recombination with Tournament Parent Selection Function

```
def discrete_recombination_tournament_parent_selection(population: Population)
    -> Individual:
    parent1, parent2 = population.k_tournament_selection(2)
    offspring = parent1.discrete_recombination(parent2)
    return offspring
```

Listing 9: Discrete Recombination with Roulette Parent Selection Function

```
def discrete_recombination_roulette_parent_selection(population: Population)
    -> Individual:
    parent1, parent2 = population.roulette_selection(), population.
        roulette_selection()
    offspring = parent1.discrete_recombination(parent2)
    return offspring
```

Listing 10: Intermediate Recombination with Random Parent Selection Function

```
def intermediate_recombination_random_parent_selection(population: Population)
    -> Individual:
    parent1, parent2 = population.random_selection(), population.
        random_selection()
    offspring = parent1.intermediate_recombination(parent2)
    return offspring
```

Listing 11: Intermediate Recombination with Tournament Parent Selection Function

```
def intermediate_recombination_tournament_parent_selection(population:
    Population) -> Individual:
    parent1, parent2 = population.k_tournament_selection(2)
    offspring = parent1.intermediate_recombination(parent2)
    return offspring
```

Listing 12: Intermediate Recombination with Roulette Parent Selection Function

```
def intermediate_recombination_roulette_parent_selection(population:
    Population) -> Individual:
    parent1, parent2 = population.roulette_selection(), population.
        roulette_selection()
    offspring = parent1.intermediate_recombination(parent2)
    return offspring
```

B.3 Offspring Mutation

Listing 13: Offspring Mutation

```
def mutate(self, random_state: int = None):
        """
        Perform mutation on the individual.

        Args:
        - random_state (int): Seed for the random number generator. Defaults
            to None.

        Returns:
        - Individual: The mutated individual.
        """
        rs = np.random.RandomState(
            random_state) if random_state is not None else np.random
        n = len(self.search_variables)

        # Learning rates
        lr_global = 1 / np.sqrt(2 * n)  # t'
        lr_local = 1 / np.sqrt(2 * np.sqrt(n))  # t

        # Sample a global perturbation
        g = rs.normal(0, 1)
```

```
# Mutate individual step sizes
sigma_prime = [
    sig * np.exp(lr_global * g + lr_local * rs.normal(0, 1)) for sig
        in self.strategy_parameters]

# Mutate search variables
x_prime = [xi + sigpi * rs.normal(0, 1)
           for xi, sigpi in zip(self.search_variables, sigma_prime)]

return Individual(x_prime, sigma_prime)
```